

AquaGuardian Regression System - Complete Documentation

Table of Contents

- 1. [System Overview](#)
- 2. [Main Flow & Architecture](#)
- 3. [Core Components](#)
- 4. [Linear Regression Components](#)
- 5. [Trial System Components](#)
- 6. [Data Models & Utilities](#)
- 7. [Main Function Flow](#)
- 8. [Usage Examples](#)

System Overview

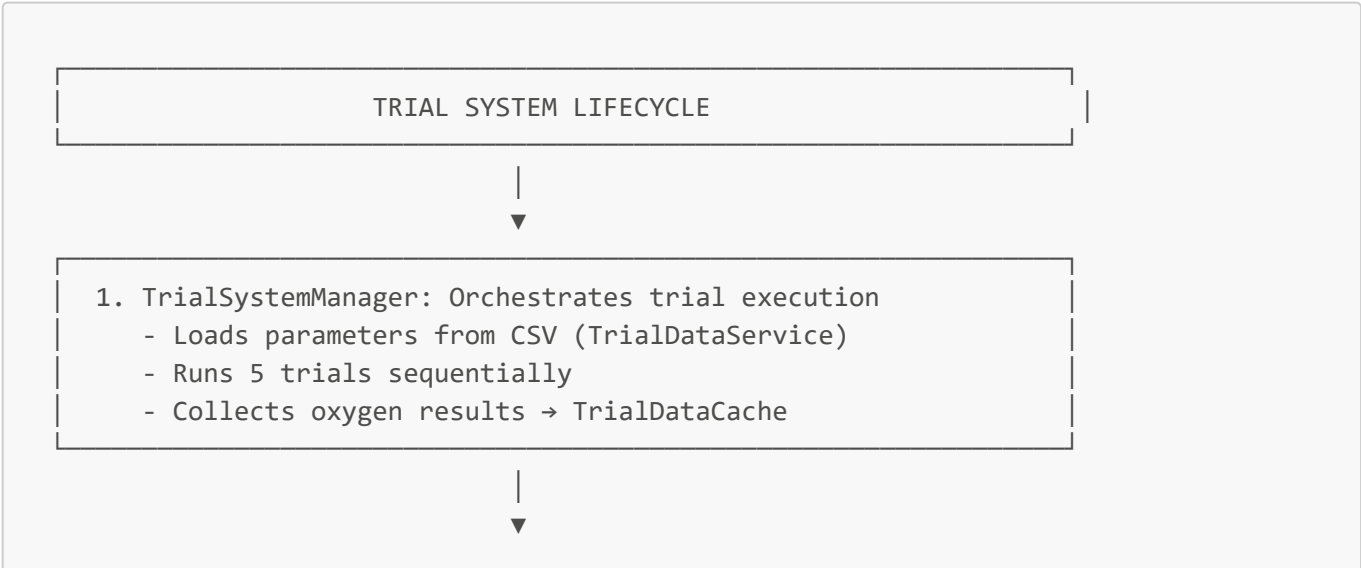
The AquaGuardian Regression System is an **Active Learning** framework that uses **Machine Learning (Multiple Linear Regression)** to:

- 1. **Predict** player oxygen levels based on game parameters
- 2. **Optimize** game difficulty parameters to achieve target oxygen levels (default: 10%)
- 3. **Adapt** difficulty automatically based on player performance across trials

Key Features

- **Small-n Regression:** Optimized for 3-10 trials (uses Ridge regularization, adaptive feature selection)
- **Real-time Optimization:** 3-phase solver (analytical → gradient descent → iterative refinement)
- **Cross-validation:** K-Fold CV for model validation
- **Comprehensive Reporting:** Detailed analysis with feature importance, coefficients, and predictions

Main Flow & Architecture



2. TrialRegressionAlgorithm: Main regression coordinator

INPUT: List<TrialData> (5 trials with oxygen results)

OUTPUT: RegressionResult (optimized parameters + report)



3. REGRESSION PIPELINE (coordinated by RegressionUtilities)

Step 1: Feature Extraction (FeatureExtractor)

- Extracts 10 features from TrialData
- Handles Amadeo/Keyboard mode differences

Step 2: Model Training (OxygenPredictor)

- Trains MultipleLinearRegression model
- Applies normalization (FeatureNormalizer)
- Uses Ridge regularization for stability

Step 3: Cross-Validation

- K-Fold CV (2-5 folds based on sample size)
- Calculates RMSE, MAE, R²

Step 4: Parameter Optimization (DifficultyParameterSolver)

- Finds optimal parameters for target oxygen (10%)
- 3-phase approach: analytical → gradient → iterative
- Fallback: RandomSweepOptimizer if needed

Step 5: Report Generation (TrialReportGenerator)

- Summary report (5-8 lines)
- Full detailed report with analysis
- Saves to file (RegressionResults folder)

Core Components

1. TrialRegressionAlgorithm.cs

Main orchestrator for the entire regression pipeline

Main Function: PerformRegressionAnalysis()

```
public static RegressionResult PerformRegressionAnalysis(  
    List<TrialData> allTrialData,  
    float targetOxygen = 10f)
```

Flow:

- 1. **Validation:** Checks if at least 3 trials available

2. **Statistics:** Calculates average oxygen across trials
3. **Training:** Trains OxygenPredictor model
4. **Cross-Validation:** Performs K-Fold CV for validation
5. **Feature Preparation:** Filters features (removes derived features like EffectiveDrainRate)
6. **Optimization:** Calls `RegressionUtilities.OptimizeParameters()` to find optimal parameters
7. **Report Generation:** Creates summary and full reports
8. **Return:** RegressionResult with optimized parameters and analysis

Key Constants:

- `DEFAULT_TARGET_O2 = 10f` - Target oxygen level (10%)
 - `TARGET_TOLERANCE = 5f` - Acceptable error range ($\pm 5\%$)
 - `RANDOM_SWEEP_THRESHOLD = 5.0f` - Error threshold for fallback to random search
-

2. RegressionUtilities.cs

Helper methods for regression pipeline

Main Function: `OptimizeParameters()`

```
public static (TrialData solution, float error) OptimizeParameters(  
    MultipleLinearRegression model,  
    OxygenPredictor predictor,  
    List<TrialData> trials,  
    (string feature, float importance)[] filteredImportance,  
    string[] featureNames,  
    float targetOxygen,  
    bool anyAmadeoTrials)
```

3-Phase Optimization Strategy:

```
Phase 1: SolveForTargetOxygen (Primary - Most Accurate)  
├─ Analytical minimal-change solution  
├─ Gradient descent refinement  
├─ Iterative refinement if error > 0.5%  
└─ Typical error: < 1%  
  
Phase 2: RandomSweepOptimizer (Fallback if error > 5%)  
├─ Random parameter search (150 samples)  
└─ Returns best candidate found  
  
Phase 3: SolveForTargetDifficultyMulti (Final Fallback)  
├─ Gradient descent on top features  
└─ Adaptive learning rate based on error
```

Other Key Functions:

- `CalculateTrialStatistics()` - Calculates average oxygen
 - `PerformCrossValidationAndErrorCalculation()` - K-Fold CV
 - `PrepareOptimizationFeatures()` - Filters banned features (EffectiveDrainRate, factorForce in keyboard mode)
 - `ConstrainRangesToObserved()` - Constrains parameter ranges based on observed data
-

3. TrialDataCache.cs

Caches oxygen values across multiple runs

Purpose:

- Stores oxygen results from completed trials
- Supports multiple runs (each run = 5 trials)
- Provides data for regression analysis

Key Methods:

```
BeginRun()                // Start new run
AppendTrial(trialId, oxygen) // Add trial result
EndRun()                  // Finalize run
GetLatestRunOxygenValues() // Get last run's data (5 values)
GetAllRunsHistory()       // Get all runs
```

Data Structure:

- `currentRunOxygenValues`: Dictionary<int, float> - Current run data
 - `allRunsHistory`: List<List> - All completed runs
 - Fixed size: 5 trials per run
-

4. TrialDataModels.cs

Data models for the system

TrialData (Main data structure)

```
public class TrialData {
    // Identifiers
    int trialId;

    // Game Parameters (9 base parameters)
    float speed;                // Forward speed
    float verticalSpeed;        // Vertical movement speed
    float idleUpwardSpeed;      // Passive upward drift
    float lifeTime;             // Oxygen tank lifetime (seconds)
    float RemoveHealthEveryLifeTime; // Health removed per cycle
```

```

float removeHealthWithCollide;    // Collision damage
float timeBetweenCollides;       // Collision cooldown
float healHealthPoint;           // Health pack restoration
float factorForce;               // Amadeo force multiplier

// Input Mode
float IsAmadeoMode;              // 1 = Amadeo, 0 = Keyboard

// Results
float finalOxygenRemaining;       // Final oxygen % (target)
bool completed;                  // Trial completion status
float trialDuration;              // Duration in seconds

// Derived Properties (calculated, not stored)
float EffectiveDrainRate =>       // RemoveHealthEveryLifeTime / lifeTime
    RemoveHealthEveryLifeTime / Max(0.1f, lifeTime);
}

```

RegressionResult

```

public class RegressionResult {
    string summaryText;            // Short summary (5-8 lines)
    string fullDetailsText;        // Full detailed report
    Dictionary<string, float> correlations; // Feature importance
    float averageOxygen;           // Average across trials
    int totalTrials;               // Number of trials analyzed
    List<TrialData> analyzedTrials; // All trial data
    TrialData optimizedSolution;    // Optimized parameters
    float optimizedSolutionError;  // Prediction error (%)
}

```

ParameterRanges

```

public class ParameterRanges {
    Vector2 speedRange = (10f, 40f);
    Vector2 verticalSpeedRange = (15f, 45f);
    Vector2 idleUpwardSpeedRange = (0.01f, 3f);
    Vector2 healHealthPointRange = (3f, 15f);
    Vector2 timeBetweenCollidesRange = (1f, 5f);
    Vector2 removeHealthWithCollideRange = (5f, 20f);
    Vector2 RemoveHealthEveryLifeTimeRange = (0.5f, 7f);
    Vector2 lifeTimeRange = (0.5f, 4f);
    Vector2 factorForceRange = (0.5f, 5f);
}

```

Linear Regression Components

5. MultipleLinearRegression.cs

Core ML model - Multiple Linear Regression with Ridge regularization

Model Equation:

$$y = \beta_0 + \beta_1 \cdot x_1 + \beta_2 \cdot x_2 + \dots + \beta_k \cdot x_k$$

Where:

- y = predicted oxygen %
- β_0 = intercept
- β_i = coefficients (feature weights)
- x_i = normalized features

Key Features:

- **Normalization:** Standardizes features to mean=0, std=1
- **Ridge Regularization:** Adds $\lambda \cdot \|\beta\|^2$ penalty to prevent overfitting
 - Default $\lambda = 0.5$ (adaptive: 0.5-2.0 based on sample size)
 - Does NOT penalize intercept β_0
- **Cholesky Decomposition:** Efficient solver for positive definite matrices

Main Methods:

```
Fit(X, Y, featureNames)           // Train model
Predict(features)                  // Predict oxygen for single trial
PredictBatch(X)                    // Predict for multiple trials
KFoldCV(X, Y, kFolds)              // K-Fold cross-validation
GetFeatureImportance()             // Get feature importance ( $|\beta_i|$ )
```

Metrics Calculated:

- **R^2** (R-squared): Proportion of variance explained (0-1, higher = better)
- **Adjusted R^2** : R^2 adjusted for number of features
- **RMSE** (Root Mean Squared Error): Average prediction error
- **MAE** (Mean Absolute Error): Average absolute error

6. OxygenPredictor.cs

High-level wrapper for MultipleLinearRegression

Purpose:

- Simplifies model training and prediction
- Handles feature selection for small datasets (<10 trials)

- Applies adaptive Ridge regularization

Adaptive Regularization:

```
float adaptiveRidgeLambda = Clamp(
    0.5f + (10f - trialCount) * 0.2f, // More regularization for fewer trials
    0.5f, // Min: 0.5
    2.0f // Max: 2.0
);
```

Feature Selection (for small datasets):

- If trials < 10 and `enableFeatureSelection = true`:
 1. Train temporary model on all features
 2. Select top K features by importance (K = `maxFeaturesForTraining`)
 3. Retrain model on selected features only
 4. Reduces overfitting risk

Main Methods:

```
TrainModel(trials, enableFeatureSelection) // Train predictor
PredictOxygen(parameters)                 // Predict with clamping [0, 100]
PredictOxygenUnclamped(parameters)         // Predict without clamping
GetFeatureImportance()                     // Get feature importance
GetModel()                                 // Get underlying MLR model
```

7. DifficultyParameterSolver.cs

Finds optimal parameters to achieve target oxygen level

Three Main Solvers:

A. `SolveForTargetOxygen()` (Primary - Most Accurate)

3-Phase Optimization:

Phase 1: Minimal Change (Analytical)

```
Goal: Find minimal parameter changes to reach target
Method: Solve linear system analytically
- Calculate contribution of fixed features
- Scale free features proportionally to coefficients
- Minimize ||Δx||² subject to f(x) = target
```

Phase 2: Gradient Refinement

Goal: Refine solution using gradient descent
Method: Projected gradient descent

- Update: $x \leftarrow x - \alpha \cdot \nabla f(x)$
- Adaptive learning rate based on error
- Clamp to valid parameter ranges

Phase 3: Iterative Refinement (if error > 0.5%)

Goal: Extended refinement with best-solution tracking
Method: Iterative gradient descent

- Tracks best solution found
- Adaptive learning rate (0.05-0.7)
- Stops when error < 0.1% or no improvement

Typical Performance:

- Error: < 1% (often < 0.5%)
- Iterations: 50-400 (adaptive based on initial error)

B. RandomSweepOptimizer() (Fallback)

Goal: Random search when gradient methods fail
Method: Monte Carlo sampling

- Generate 150 random parameter combinations
- Predict oxygen for each
- Return best candidate (lowest error)

When Used:

- Primary solver error > 5%
- Data far from target (>40% difference)
- Non-convex optimization landscape

C. SolveForTargetDifficultyMulti() (Final Fallback)

Goal: Gradient descent on top features
Method: Normalized gradient descent

- Update top K features (K = 3-5)
- Normalize by sum of squared coefficients
- Adaptive learning rate based on error distance

When Used:

- Both previous methods failed or error > 5%
- Provides guaranteed solution (may have higher error)

8. FeatureExtractor.cs

Extracts features from TrialData for ML model

10 Features Extracted:

```

1. speed                // Forward speed
2. verticalSpeed        // Vertical movement speed
3. idleUpwardSpeed      // Passive upward drift
4. lifeTime             // Oxygen tank lifetime
5. RemoveHealthEveryLifeTime // Health removed per cycle
6. removeHealthWithCollide // Collision damage
7. timeBetweenCollides  // Collision cooldown
8. healHealthPoint      // Health pack restoration
9. factorForce          // Amadeo force multiplier
10. EffectiveDrainRate   // Derived: RemoveHealthEveryLifeTime / lifeTime

```

Input Mode Adjustments:

```

// Amadeo Mode (IsAmadeoMode = 1)
idleUpwardSpeed *= 0.5f // Weaker drift for Amadeo
factorForce = actual value // Force multiplier active

// Keyboard Mode (IsAmadeoMode = 0)
idleUpwardSpeed = unchanged // Normal drift
factorForce = 0f // No force sensitivity

```

Main Methods:

```

ExtractFeatures(trial) // Single trial → float[]
ExtractFeatures(trials) // Multiple trials → float[][]
ExtractFeaturesAndTargets(trials) // Returns (X, y) for training
GetPatientBaseline(trials, ranges, useMedian) // Get personalized baseline
GetMidRangeDefaults(ranges) // Get default mid-range values

```

9. FeatureNormalizer.cs

Standardizes features to mean=0, std=1

Normalization Formula:

$$z = (x - \mu) / \sigma$$

Where:

- z = normalized value
- x = raw value
- μ = mean
- σ = standard deviation (sample std, $n-1$)

Why Normalize?

1. **Scale Invariance:** Features on different scales (e.g., speed=30, lifeTime=2) → equal importance
2. **Faster Convergence:** Gradient descent converges faster
3. **Numerical Stability:** Prevents overflow/underflow in calculations

Main Methods:

```
Fit(X)                // Calculate  $\mu$  and  $\sigma$  from training data
Transform(X)           // Normalize:  $(X - \mu) / \sigma$ 
TransformSample(x)     // Normalize single sample
InverseTransform(Z)    // Denormalize:  $Z \cdot \sigma + \mu$ 
FitTransform(X)        // Fit and transform in one call
```

10. RegressionMath.cs

Low-level mathematical operations

Key Functions:

Matrix Operations:

```
SolveSPDByCholesky(A, b) // Solve  $Ax=b$  using Cholesky decomposition
                          // For symmetric positive definite matrices
                          //  $O(n^3)$  complexity, numerically stable

CleanMatrix(X)           // Replace NaN/Inf with 0
CleanVector(y)           // Replace NaN/Inf with 0
ValidateInputs(X, Y)     // Check for null, empty, dimension mismatch
```

Metrics:

```
ComputeMetrics(yTrue, yPred) // Calculate RMSE, MAE,  $R^2$ 
```

Helper Functions:

```

EffectiveBeta(model, featureIdx, trial, optimizingDrainRateDeps)
    // Get effective coefficient for feature
    // Handles derived features (EffectiveDrainRate)
    // Accounts for normalization

SumBetaSq(model, freeFeatures, trial)
    // Sum of squared coefficients for free features
    // Used for gradient normalization

SubsetX(X, indices)           // Extract subset of rows from X
SubsetY(Y, indices)           // Extract subset of elements from Y

```

Trial System Components

11. TrialSystemManager.cs

Manages trial system lifecycle**Trial Lifecycle:**

```

1. StartTrials()
   |─ Initialize trial mode
   |─ Disable finish triggers
   |─ Start first trial

2. For each trial (1-5):
   |─ Load parameters from CSV (TrialDataService)
   |─ Build cave from CSV (PanelOpenUp)
   |─ Spawn fish (TrialFishSpawner)
   |─ Reset player position (GameSystemResetter)
   |─ Start trial timer
   |─ Player plays...
   |─ OnGameEnded() triggered

3. OnGameEnded(finalOxygen, completed)
   |─ Calculate trial duration
   |─ Detect input mode (Amadeo vs Keyboard)
   |─ Save result to CSV (TrialParameterManager)
   |─ Cache oxygen value (TrialDataCache)
   |─ Show trial control panel (TrialUIController)
   |─ Wait for user action (Continue/Restart)

4. ContinueToNextTrial()
   |─ If trial < 5: Start next trial
   |─ If trial = 5: CompleteAllTrials()

```

```
5. CompleteAllTrials()
  └─ Exit trials mode
  └─ Cleanup trial objects
  └─ Show completion UI with regression analysis
```

Key Methods:

```
StartTrials()                // Initialize trial system
ContinueToNextTrial()        // Advance to next trial
RestartCurrentTrial()        // Restart current trial
OnTrialFishReached(oxygen, completed) // Handle trial end
CompleteAllTrials()          // Finalize all trials
```

12. TrialDataService.cs

Centralized CSV operations for trial data

Main Functions:

Loading:

```
LoadTrialParameters(trialId, useRandom, customPath)
  // Load single trial parameters by ID
  // Used during gameplay to get parameters for current trial

LoadTrialDataFromCache(customPath)
  // Load trials with oxygen from cache
  // Falls back to CSV if cache incomplete
  // Used for regression analysis

LoadAllTrials(useRandom, customPath)
  // Load all trials with oxygen from CSV
  // Used for regression when cache not available
```

Saving:

```
SaveTrialResult(trialData, isRandomMode, customTrialPath, customRandomPath)
  // Save trial result back to CSV
  // Regular mode: Adds new o2_run column
  // Random mode: Updates o2_result column
```

Generation:

```
GenerateRandomParameters(ranges)
// Generate random parameters within ranges
// Used for random parameter mode
```

CSV Formats:

Regular CSV (Trial_5_runs_.csv):

```
trialId,speed,verticalSpeed,...,factorForce,IsAmadeoMode,o2_run1,o2_run2,...
1,25.0,30.0,...,0.0,0.0,45.2,52.1,...
2,20.0,25.0,...,0.0,0.0,38.7,41.3,...
```

Random CSV (Trial_Random_Parameters.csv):

```
trial_id,speed,verticalSpeed,...,factorForce,IsAmadeoMode,o2_result,duration
1,28.5,32.1,...,0.0,0.0,47.3,125.4
2,22.3,27.8,...,0.0,0.0,39.1,142.7
```

13. TrialReportGenerator.cs

Generates regression analysis reports

Two Report Types:

A. Summary Report (5-8 lines)

```
=== REGRESSION ANALYSIS SUMMARY ===

Trials: 5 (5R+0Rand) | Target: 10.0% | Avg Error: 1.23%

Optimized Parameters:
speed: 25.00, verticalSpeed: 30.00, idleUpwardSpeed: 1.50, ...

Prediction: 10.15% | Error: 0.15%

Coefficients:
Intercept: 45.2341, speed: -0.8234, verticalSpeed: 0.5123, ...
```

B. Full Report (Detailed)

```
Regression | 2025-11-05 14:23:45 | 5 trials
Avg : 57.6% | Data Source: Constant Parameters CSV | Trials: [1,2,3,4,5]
```

```
(5R+0Rand)
R^2=0.996 AdjR^2=0.992 RMSE=1.21% MAE=0.93%
input type : keyboard

Raw Data:
T1: Speed=25.00 VerticalSpeed=30.00 ... FinalOxygen=58.2% Duration=125.3s
T2: Speed=20.00 VerticalSpeed=25.00 ... FinalOxygen=45.1% Duration=142.7s
...

Trial 1 Analysis:
  Actual Oxygen: 58.20%
  Predicted Oxygen: 57.85%
  Error: 0.35% (0.6%)
  Duration: 125.3s
  Feature Contributions:
    Intercept: 45.2341
    Forward speed: -0.8234 * 25.00 = -20.5850
    Vertical speed: 0.5123 * 30.00 = 15.3690
    ...

Feature Importance (Impact on Oxygen):
  RemoveHealthEveryLifeTime      0.8234
  removeHealthWithCollide        0.6123
  speed                          0.5234
  ...

Regression Coefficients:
  Intercept      45.2341
  Forward speed  -0.8234
  Vertical speed  0.5123
  ...

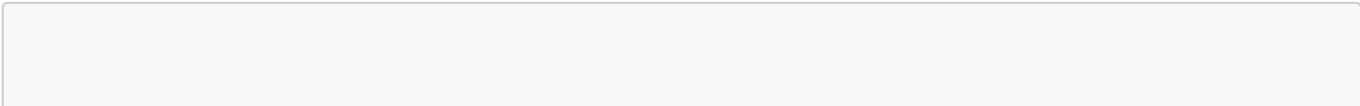
Regression Equation:
Oxygen = 45.2341
        - 0.8234 * Forward speed
        + 0.5123 * Vertical speed
        ...

OPTIMIZED PARAMETERS FOR TARGET 10% OXYGEN

Predicted: 10.15% | Error: 0.15%

Parameters:
  Speed           = 28.50
  VerticalSpeed    = 32.00
  IdleUpwardSpeed = 1.50
  ...
```

Main Methods:



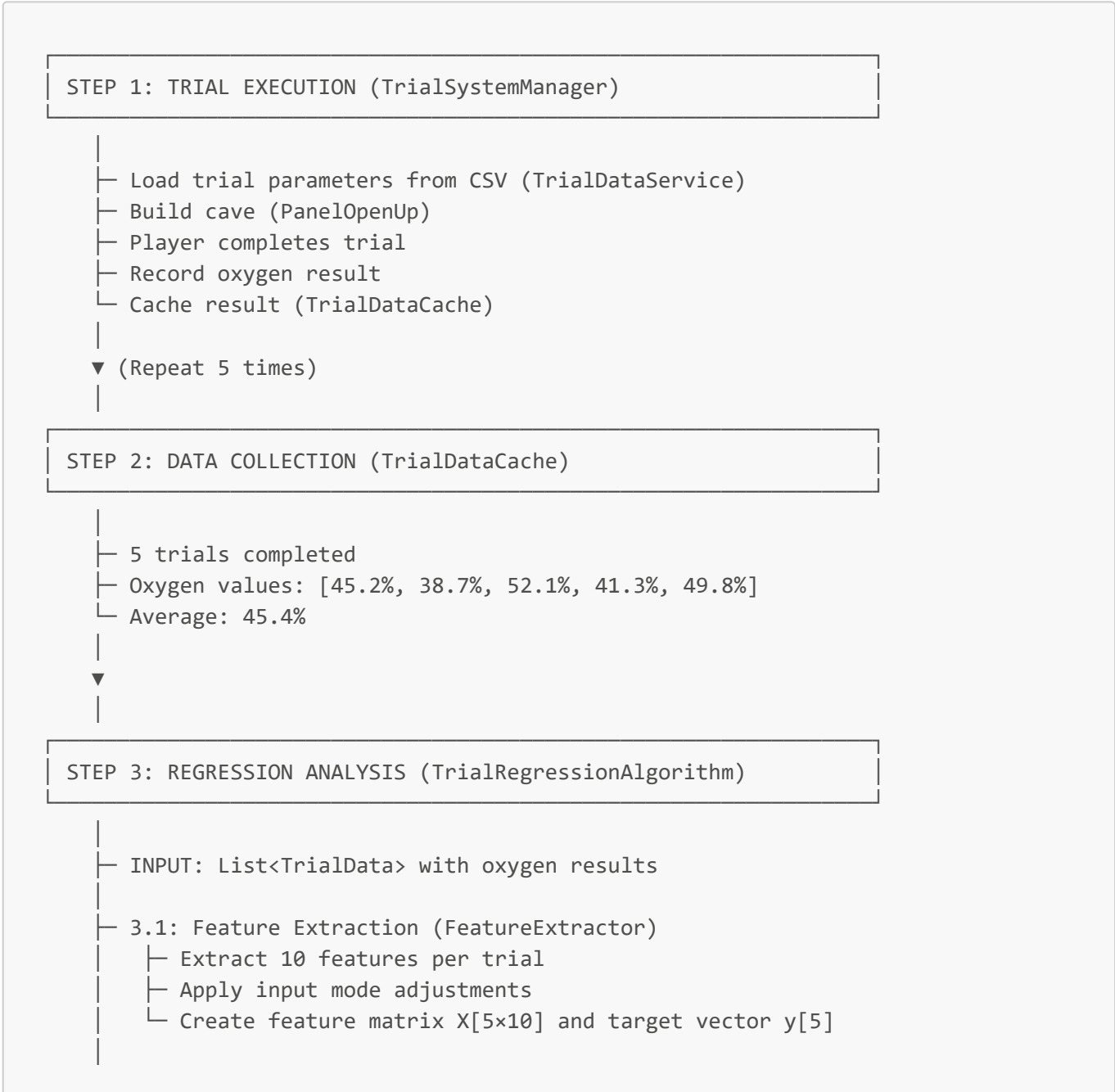
```
GenerateSummaryReport(trials, avgError, predictor, optimalParams,
                      optimizedSolution, optimizedSolutionError, targetOxygen)
    // Generate short summary (5-8 lines)

GenerateFullReport(trials, avgError, avgOxygen, predictor, optimalParams,
                  cvRmse, cvMae, cvR2, kFolds, optimizedSolution,
                  optimizedSolutionError, targetOxygen)
    // Generate detailed report with all analysis

SaveToFile(fullReport, trials, totalTrials, saveFolder)
    // Save report to file in RegressionResults folder
```

Main Function Flow

Complete Flow from Trial to Optimized Parameters



```

├─ 3.2: Model Training (OxygenPredictor → MultipleLinearRegression)
│   ├── Normalize features (FeatureNormalizer)
│   ├── Apply Ridge regularization ( $\lambda = 0.5-2.0$ )
│   ├── Solve using Cholesky decomposition
│   └── Calculate coefficients  $\beta_0, \beta_1, \dots, \beta_{10}$ 
├─ 3.3: Cross-Validation (RegressionUtilities)
│   ├── K-Fold CV ( $k = 2-5$  based on sample size)
│   ├── Calculate RMSE, MAE,  $R^2$ 
│   └── Validate model performance
├─ 3.4: Parameter Optimization (RegressionUtilities →
DifficultyParameterSolver)
│   ├── Phase 1: SolveForTargetOxygen (Primary)
│   │   ├── Analytical minimal-change solution
│   │   ├── Gradient descent refinement
│   │   ├── Iterative refinement if error > 0.5%
│   │   └── Typical error: < 1%
│   ├── Phase 2: RandomSweepOptimizer (if error > 5%)
│   │   ├── Random search (150 samples)
│   │   └── Return best candidate
│   └── Phase 3: SolveForTargetDifficultyMulti (final fallback)
│       ├── Gradient descent on top features
│       └── Adaptive learning rate
├─ 3.5: Report Generation (TrialReportGenerator)
│   ├── Generate summary report
│   ├── Generate full detailed report
│   └── Save to file
└─ OUTPUT: RegressionResult
    ├── optimizedSolution: TrialData (optimal parameters)
    ├── optimizedSolutionError: 0.15% (prediction error)
    ├── summaryText: "Trials: 5 | Target: 10.0% | ..."
    └── fullDetailsText: Complete analysis report

```

Usage Examples

Example 1: Basic Regression Analysis

```

// 1. Load trial data from cache
var trials = TrialDataService.LoadTrialDataFromCache();

// 2. Perform regression analysis
var result = TrialRegressionAlgorithm.PerformRegressionAnalysis(
    trials,
    targetOxygen: 10f

```



```
);

// 3. Display results
Debug.Log(result.summaryText);

// 4. Save to file
TrialRegressionAlgorithm.SaveRegressionResultsToFile(result);

// 5. Use optimized parameters for next trial
if (result.optimizedSolution != null)
{
    ApplyParameters(result.optimizedSolution);
}
```

Example 2: Custom Target Oxygen

```
// Target 15% oxygen instead of default 10%
var result = TrialRegressionAlgorithm.PerformRegressionAnalysis(
    trials,
    targetOxygen: 15f
);

Debug.Log($"Optimized for 15% oxygen:");
Debug.Log($"Predicted: {predictor.PredictOxygen(result.optimizedSolution)}%");
Debug.Log($"Error: {result.optimizedSolutionError}%");
```

Example 3: Manual Prediction

```
// Train predictor
var predictor = new OxygenPredictor();
predictor.TrainModel(trials, enableFeatureSelection: false);

// Create test parameters
var testParams = new TrialData {
    speed = 25f,
    verticalSpeed = 30f,
    idleUpwardSpeed = 1.5f,
    lifeTime = 2.0f,
    RemoveHealthEveryLifeTime = 3.0f,
    removeHealthWithCollide = 10f,
    timeBetweenCollides = 2.0f,
    healHealthPoint = 8f,
    factorForce = 0f,
    IsAmadeoMode = 0f
};

// Predict oxygen
float predictedOxygen = predictor.PredictOxygen(testParams);
Debug.Log($"Predicted oxygen: {predictedOxygen}%");
```

Example 4: Feature Importance Analysis

```
var predictor = new OxygenPredictor();
predictor.TrainModel(trials);

var importance = predictor.GetFeatureImportance();
Debug.Log("Feature Importance:");
foreach (var (feature, value) in importance)
{
    Debug.Log($" {feature}: {value:F4}");
}
```

Example 5: Custom Parameter Ranges

```
// Define custom ranges
var ranges = new ParameterRanges {
    speedRange = new Vector2(15f, 35f),
    verticalSpeedRange = new Vector2(20f, 40f),
    // ... other ranges
};

// Generate random parameters within ranges
var randomParams = TrialDataService.GenerateRandomParameters(ranges);

// Use for trial
ApplyParameters(randomParams);
```

Key Insights & Design Decisions

1. Why Ridge Regularization?

- **Problem:** With 5 trials and 10 features, risk of overfitting (more features than samples)
- **Solution:** Ridge adds penalty $\lambda \cdot ||\beta||^2$ to prevent large coefficients
- **Result:** More stable predictions, better generalization

2. Why Normalize Features?

- **Problem:** Features on different scales (speed=30, lifeTime=2)
- **Solution:** Standardize to mean=0, std=1
- **Result:** Equal importance, faster convergence, numerical stability

3. Why 3-Phase Optimization?

- **Phase 1 (Analytical):** Fast, accurate for convex problems
- **Phase 2 (Random):** Handles non-convex landscapes

- **Phase 3 (Gradient):** Guaranteed solution, fallback
- **Result:** Robust optimization that rarely fails

4. Why Derived Features (EffectiveDrainRate)?

- **Insight:** `RemoveHealthEveryLifeTime / lifeTime` = drain rate per second
- **Benefit:** Captures interaction between two parameters
- **Note:** Banned from optimization (derived, not independent)

5. Why Adaptive Learning Rate?

```
float adaptiveLR = baseLR * Min(1f, Abs(error) / 5f);
```

- **Far from target (error > 5%):** Large steps (fast convergence)
- **Close to target (error < 5%):** Small steps (precision)
- **Result:** Fast convergence without overshooting

Performance Characteristics

Typical Results (5 trials):

- **Training Time:** < 50ms
- **Optimization Time:** 50-200ms
- **Total Analysis Time:** < 300ms
- **Prediction Error:** < 1% (often < 0.5%)
- **R² Score:** 0.85-0.99 (excellent fit)

Scalability:

- **Minimum Trials:** 3 (barely functional)
- **Optimal Trials:** 5-10 (best balance)
- **Maximum Trials:** 100+ (supported, but diminishing returns)

Troubleshooting

Common Issues:

1. "Need at least 3 trials for ML analysis"

- **Cause:** Not enough trial data
- **Solution:** Complete at least 3 trials before running regression

2. "No variance in oxygen data"

- **Cause:** All trials have same oxygen result
- **Solution:** Vary game parameters to get different outcomes

3. "Optimization failed: error > 5%"

- **Cause:** Data far from target or non-linear relationships
- **Solution:** System automatically tries fallback methods (random search)

4. "Feature count mismatch"

- **Cause:** Model trained on different features than prediction input
 - **Solution:** Ensure consistent feature extraction (use FeatureExtractor)
-

Future Enhancements

Potential Improvements:

1. **Non-linear Models:** Neural networks for complex relationships
 2. **Online Learning:** Update model after each trial (incremental learning)
 3. **Multi-objective Optimization:** Optimize for oxygen AND completion time
 4. **Personalized Models:** Separate models per player
 5. **Confidence Intervals:** Uncertainty estimates for predictions
-

References & Related Files

Core Scripts:

- `TrialRegressionAlgorithm.cs` - Main orchestrator
- `RegressionUtilities.cs` - Helper methods
- `TrialDataModels.cs` - Data structures
- `TrialDataCache.cs` - Data caching

Linear Regression:

- `MultipleLinearRegression.cs` - ML model
- `OxygenPredictor.cs` - High-level wrapper
- `DifficultyParameterSolver.cs` - Optimization
- `FeatureExtractor.cs` - Feature engineering
- `FeatureNormalizer.cs` - Normalization
- `RegressionMath.cs` - Math utilities

Trial System:

- `TrialSystemManager.cs` - Trial lifecycle
- `TrialDataService.cs` - CSV operations
- `TrialReportGenerator.cs` - Report generation

Documentation:

- `SYSTEM_OVERVIEW.md` - Complete system documentation
- `README_SIMPLE.md` - Quick start guide

Contact & Support

For questions or issues with the regression system, refer to:

1. Code comments in each script
2. [SYSTEM_OVERVIEW.md](#) for architecture details
3. Unity console logs (detailed debugging information)

Last Updated: November 5, 2025

Version: 1.0

Author: AquaGuardian Development Team